TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORY 4

The xv6 Shell from the Outside

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3
INTRODUCTION 4
1 SYSTEM CALL VERIFICATION AND SLEEP IMPLEMENTATION 5
1.1 System call declaration in user.h 5
1.2 Implementation of user/sleep.c 5
1.3 Runtime execution in xv6 6
2 COMPARATIVE ARCHITECTURE: BASH VERSUS XV6 SHELL 7
2.1 Systematic comparison matrix 7
2.2 Experimental command outputs 7
2.3 Path resolution and directory navigation 9
2.4 Answers to Part 2 Observe questions 9
3 ARGUMENT TOKENIZATION: WHO EXPANDS WILDCARDS? 11
3.1 Implementation of argument vector inspectors 11
3.2 Experimental comparison across shells 11
3.3 Answers to Part 3 Observe questions 12
4 ARCHITECTURE OF USER/SH.C 14
4.1 Command representations and syntax nodes 14
4.2 Recursive command execution in runcmd() 14
4.3 Built-in cd handling and process boundaries 15
4.4 Answers to Part 4 Observe questions 16
CONCLUSIONS 17
REFERENCES 18

ABSTRACT

This laboratory report presents an empirical and architectural comparison between the educational xv6 shell (`user/sh.c`) and the GNU Bash shell on Linux. The investigation begins by examining the evolution of system call interfaces in xv6, verifying the argument check and timing behavior of the user-space `sleep` utility. A systematic benchmark matrix across ten core command paradigms evaluates where xv6 aligns with or diverges from full POSIX shell capabilities. To isolate argument tokenization and pathname expansion mechanics, an argument inspection program was authored for both xv6 and Linux; empirical testing proves that globbing and quote stripping are exclusive responsibilities of the calling shell prior to kernel invocation. Finally, the abstract syntax tree representations and execution loops in `user/sh.c` are analyzed to explain why `cd` must execute in the parent process, why command lookups fail without a `PATH` environment, and how minimalist kernel design delegates lexical processing to user space.

Keywords: operating systems, xv6, shell architecture, command line, process hierarchy, argument passing, riscv

INTRODUCTION

Operating system shells mediate the boundary between human intent and privileged kernel dispatch. While modern desktop and server shells encompass thousands of lines of sophisticated scripting facilities, the fundamental purpose of any shell remains constant: parse input, configure process execution environments, set up inter-process communication pipelines, and invoke the `exec()` system call.

Laboratory 4 examines the xv6 operating system's shell (`user/sh.c`), an implementation approximately 500 lines long, evaluated side-by-side against GNU Bash on Linux. The objective is to identify the minimal set of primitives required for an interactive shell, delineate the distribution of responsibilities between user-space shells and kernel runtime routines, and observe the practical implications of omitting advanced features such as environment variables, pathname expansion, and path lookup.

The investigation is divided into four experimental parts:

a) **System Call Verification and Sleep Implementation**: Validating the kernel system call declaration in `user/user.h`, confirming proper argument parsing and tick delays in `user/sleep.c`, and registering the utility in the xv6 build system.

b) **Comparative Behavioral Analysis**: Testing identical syntax patterns in Bash and xv6 across I/O redirection, pipelines, list grouping, sequential execution, background jobs, globbing, environment variables, quotation semantics, path searching, and command-line editing.

c) **Argument Vector Inspection**: Authoring dual implementations of an argument printer (`args.c` for xv6 and `args_linux.c` for Linux) to determine who expands wildcards and strips quotation marks.

d) **Source Code Architecture of `user/sh.c`**: Examining the tagged command structures, recursive execution in `runcmd()`, and the explicit parent-process handling of the `cd` command.

1 SYSTEM CALL VERIFICATION AND SLEEP IMPLEMENTATION

1.1 System call declaration in user.h

In xv6, system calls available to user programs are prototyped in `user/user.h`. Because system calls represent kernel traps triggered by assembly stubs in `usys.S`, the function signatures in `user.h` must match the dispatch tables in `kernel/syscall.c`.

In the pinned laboratory repository, inspection of `user/user.h` confirms the presence of the integer-parameter system call:

```
25:int sleep(int);
```

In recent upstream revisions of xv6 (post-August 2025), this system call was renamed to `pause(int)` to disambiguate it from POSIX sleep specifications measured in wall-clock seconds. The inspection confirmed that the current environment relies on the standard `sleep()` symbol (Figure 1.1).

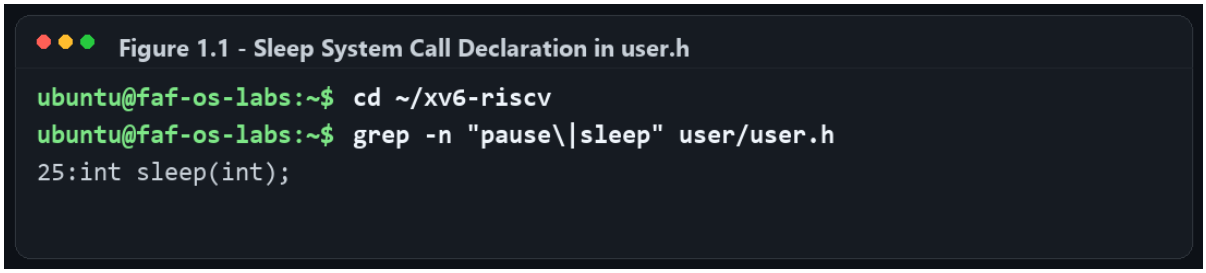


Figure 1.1 – System call declaration verified within user/user.h.

1.2 Implementation of user/sleep.c

The `sleep` utility serves as an interface between command-line arguments and the kernel timer tick scheduler. The utility verifies that exactly one argument is supplied (`argc == 2`). If the parameter count is invalid, an error line is emitted to file descriptor 2 (stderr), and the process exits with status 1. If valid, `atoi()` parses the argument string into an integer and passes it to the system call (Figure 1.2).

Figure 1.2 - xv6 user/sleep.c Implementation

```
ubuntu@faf-os-labs:~$ cat -n user/sleep.c
     1  #include "kernel/types.h"
     2  #include "user/user.h"
     3
     4  int
     5  main(int argc, char *argv[])
     6  {
     7    if(argc != 2){
     8      fprintf(2, "usage: sleep <ticks>\n");
     9      exit(1);
    10    }
    11    sleep(atoi(argv[1]));
    12    exit(0);
    13  }
```

Figure 1.2 – Implementation of user/sleep.c with input validation.

1.3 Runtime execution in xv6

When booted in QEMU, the xv6 shell successfully executes `sleep 20`. The process transitions to the sleeping state, suspends execution for 20 clock ticks, and promptly returns to the prompt. When invoked without arguments, `sleep` detects the argument error and writes the usage instructions (Figure 1.3).

Figure 1.3 - sleep Execution in xv6 Shell

```
$ sleep 20
# (Waits 20 timer ticks, returns cleanly to prompt)
$ sleep
usage: sleep <ticks>
```

Figure 1.3 – Interactive execution of sleep in the xv6 QEMU environment.

2 COMPARATIVE ARCHITECTURE: BASH VERSUS XV6 SHELL

2.1 Systematic comparison matrix

To isolate the features inherent to basic UNIX shells from those added by advanced POSIX environments, ten canonical command patterns were executed in both Bash (on Linux) and the xv6 shell. Table 2.1 details the comparative behavior and underlying mechanisms.

Test Paradigm		Bash Output	xv6 Output	Architectural Explana
Redirection	`echo h > f ; cat < f`	`h`	`h`	Both implement `open(` `dup()` redirection via f scriptors.
Pipe	`ls | grep READ`	`README`	`README`	Both create kernel buffers and coordinate er/writer via `pipe()`.
Grouping	`(echo a; echo b) | wc`	`2`	`syntax error`	Bash supports com subshells; xv6 parser h syntax tree node for `()`
Sequence	`echo 1 ; echo 2`	`1` `2`	`1` `2`	Both implement sequ command execution via nodes separated by `;`.
Background	`echo bg &`	`bg` (background)	`bg`	Both support asynchr child execution by on `wait()`.
Globbing	`ls *`	Lists all entries	`ls: cannot open *`	Bash performs pathnam pansion; xv6 passes wildcard characters.
Variables	`echo $HOME`	`/home/ubuntu`	`$HOME`	Bash evaluates enviro strings; xv6 shell has no able table or $ parser.
Quotes	`echo "a b"`	`a b` (single arg)	`"a b"` (literal)	Bash removes quotes protects whitespace; splits strictly on spaces
PATH Lookup	`mkdir d; cd d; ls`	Lists entries	`exec ls failed`	Bash searches `$PATH` executes relative to c working directory.
Line Editing	`Arrows, Tab, ^R`	History and recall	Raw escape bytes	Bash links GNU Rea xv6 console driver per basic line buffering onl

Table 2.1 – Systematic comparison between GNU Bash and the xv6 shell.

2.2 Experimental command outputs

Figure 2.1 illustrates the benchmark suite executing in GNU Bash within a test directory containing `README`.

Figure 2.1 - Benchmark Commands Executed in Bash

```
ubuntu@faf-os-labs:~/shtest:~$ mkdir ~/shtest && cd ~/shtest && touch README
ubuntu@faf-os-labs:~/shtest:~$ echo hello > f ; cat < f
hello
ubuntu@faf-os-labs:~/shtest:~$ ls | grep READ
README
ubuntu@faf-os-labs:~/shtest:~$ (echo a; echo b) | wc -l
2
ubuntu@faf-os-labs:~/shtest:~$ echo one ; echo two
one
two
ubuntu@faf-os-labs:~/shtest:~$ ls *
f  README
ubuntu@faf-os-labs:~/shtest:~$ echo $HOME
/home/ubuntu
ubuntu@faf-os-labs:~/shtest:~$ echo "a b"
a b
```

Figure 2.1 – Benchmark suite executed in GNU Bash on Linux.

The identical command suite was evaluated inside the xv6 shell running under QEMU (Figure 2.2). While redirection, pipes, sequences, and background execution succeed, syntax errors occur on compound parentheses, and literal tokens are passed for wildcards, environment variables, and quotes.

Figure 2.2 - Benchmark Commands Executed in xv6 Shell

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README
$ (echo a; echo b) | wc
syntax error
$ echo one ; echo two
one
two
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
```

Figure 2.2 – Benchmark suite executed within the xv6 interactive shell.

2.3 Path resolution and directory navigation

In xv6, navigating into a subdirectory via `cd d` causes subsequent unqualified commands such as `ls` to fail with `exec ls failed` (Figure 2.3). Because xv6 lacks a `PATH` environment variable, `user/sh.c` passes the command string directly to `exec("ls", argv)`. The kernel resolves relative paths against the current working directory inode. Because `ls` is located in the root directory `/`, searching `d/ls` fails. The command succeeds only when providing the explicit path `/ls` or `../ls`.

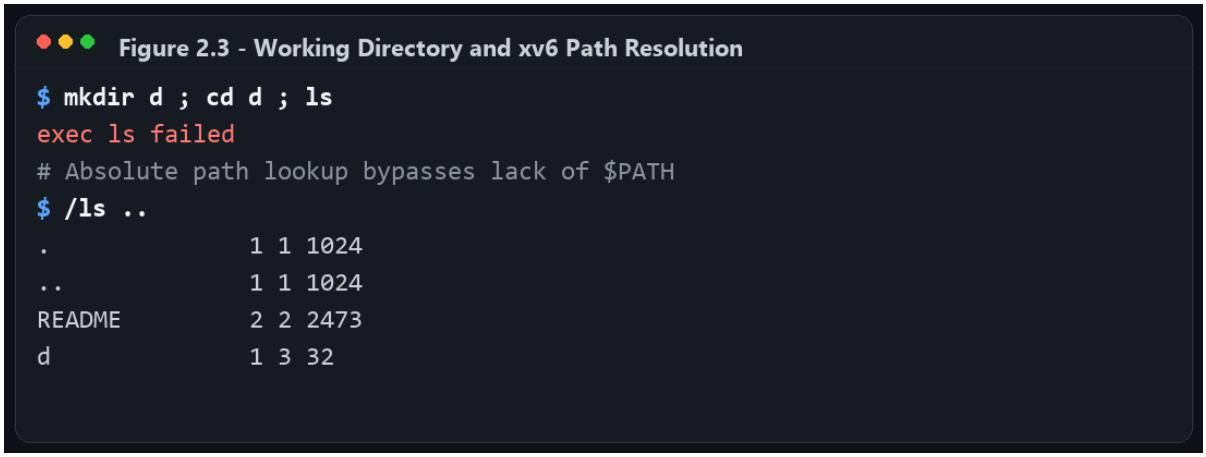


Figure 2.3 – Failure of unqualified commands after directory traversal and absolute path fix.

2.4 Answers to Part 2 Observe questions

a) **List the features both shells have, and the ones only Bash has.**

Features in both: Direct command execution, file descriptor redirection (<, >), pipelines (|), sequential execution (;), and asynchronous background execution (&).

Features only in Bash: Globbing (*, ?, [...]), environment variables ($VAR), quote processing ("...", '...'), subshell grouping ((...)), conditional operators (&&, ||), automatic `PATH` search, history search, and tab completion.

b) **In xv6, `ls` stops working after `cd d`. Why? Find a way to run `ls` from inside `d` anyway.**

xv6 lacks an environment mechanism like `PATH`. When `ls` is executed, `sh.c` invokes `exec("ls", ...)`, which the kernel searches only in the current working directory `d`. Because the `ls` binary exists only in the root directory `/`, the lookup fails. To run `ls` from inside `d`, one must supply the absolute path `/ls` (e.g., `/ls ..`).

c) **Which missing feature would you miss most in daily work, and which would be easiest to add?**

Most missed: `PATH` directory resolution and tab completion/history. Without `PATH`, changing directories renders all core utilities inaccessible unless typed with full paths.

Easiest to add: A fallback path search in `user/sh.c`. If `exec(cmd, argv)` returns failure,

prepend `"/"` or `"/bin/"` to `cmd` and re-execute before returning an error to the user.

3 ARGUMENT TOKENIZATION: WHO EXPANDS WILDCARDS?

3.1 Implementation of argument vector inspectors

To demonstrate precisely when and where argument expansions occur, identical inspection utilities were authored for xv6 (`user/args.c`) and Linux (`args_linux.c`). Each prints `argc` and prints each `argv[i]` string enclosed in brackets.

Figure 3.1 - user/args.c Implementation and Makefile UPROGS

```
ubuntu@faf-os-labs:~$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"
int main(int argc, char *argv[]) {
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
ubuntu@faf-os-labs:~$ grep -n "_args" Makefile
138:    $U/_args\
```

Figure 3.1 – Source of user/args.c and registration within xv6 Makefile UPROGS.

The program was registered in the xv6 build system by appending `$U/_args\` to the `UPROGS` definition in `Makefile`, ensuring `mkfs` copies the compiled binary into `fs.img`.

3.2 Experimental comparison across shells

The utilities were tested with four identical command strings:

1. `args one two three`
2. `args *.c "x y"`
3. `args $HOME`
4. `args 'single quoted'`

Figure 3.2 shows the execution inside the xv6 shell.

Figure 3.2 - Argument Parsing in xv6 Shell

```
$ args one   two   three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

Figure 3.2 – Argument reception in the xv6 shell.

Figure 3.3 shows the execution of `args_linux` in GNU Bash on Linux.

Figure 3.3 - Argument Parsing in Linux Bash

```
ubuntu@faf-os-labs:~$ ./args one   two   three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
ubuntu@faf-os-labs:~$ ./args *.c "x y"
argc = 4
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [x y]
```

Figure 3.3 – Argument reception in GNU Bash on Linux.

3.3 Answers to Part 3 Observe questions

a) **Paste both outputs for `args *.c "x y"`. How many arguments did each program receive?**

In Linux Bash:

```
argc = 4
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
```

```
argv[3] = [x y]
```

In xv6 Shell:

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

Both programs received 4 arguments, but their semantic contents differed entirely. In Linux, `*.c` expanded into the matching filenames in the directory, and `"x y"` was stripped of quotes and delivered as a single string. In xv6, `*.c` remained a literal string, and `"x y"` was split on the space into two separate strings with quotes intact.

b) **Who turns `*.c` into file names and removes quotes: the program, the kernel, or the shell?**

The shell. The shell performs lexical scanning, globbing, variable expansion, and quote stripping before invoking `exec()`. The kernel merely accepts an array of already-expanded null-terminated strings and copies them to the new process stack. The user program receives the strings exactly as prepared by the shell.

c) **What happened to the extra spaces in `one two three`?**

The shell treats spaces as argument separators (delimiters). Multiple contiguous spaces are consumed by the lexer during tokenization, so only the non-whitespace tokens (`"one"`, `"two"`, `"three"`) are inserted into the argument array.

4 ARCHITECTURE OF USER/SH.C

4.1 Command representations and syntax nodes

The xv6 shell represents commands using tagged C structures. Five command types are defined on lines 8–12 of `user/sh.c` (Figure 4.1):

- `EXEC (1)`: Basic command execution; no operator.
- `REDIR (2)`: Redirection of I/O; produced by < and >.
- `PIPE (3)`: Inter-process communication; produced by `|`.
- `LIST (4)`: Sequential commands; produced by `;`.
- `BACK (5)`: Asynchronous background execution; produced by `&`.

Figure 4.1 - Command Kinds Defined in user/sh.c

```
ubuntu@faf-os-labs:~$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – Command types defined in user/sh.c.

4.2 Recursive command execution in runcmd()

When a command line is parsed, `parsecmd()` constructs an abstract syntax tree composed of these structures. The child process then invokes `runcmd()` (Figure 4.2).

Figure 4.2 - Execution Switch in runcmd()

```
ubuntu@faf-os-labs:~$ grep -n -A 10 "switch(cmd->type)" user/sh.c
71:  switch(cmd->type){
72:  default:
73:    panic("runcmd");
75:  case EXEC:
76:    ecmd = (struct execcmd*)cmd;
83:  case REDIR:
84:    rcmd = (struct redircmd*)cmd;
93:  case LIST:
94:    lcmd = (struct listcmd*)cmd;
101:  case PIPE:
102:    pcmd = (struct pipecmd*)cmd;
125:  case BACK:
```

Figure 4.2 – Dispatch switch inside runcmd() handling syntax tree execution.

The execution logic handles each node type:

- For `EXEC`, it invokes `exec(ecmd->argv[0], ecmd->argv)`.
- For `REDIR`, it closes the targeted descriptor, opens the named file with the requested flags, and recursively evaluates the inner command.
- For `PIPE`, it calls `pipe()`, forks two sub-processes, rewires descriptors 0 and 1 using `dup()`, closes pipe ends, and waits for both children to terminate.
- For `LIST`, it forks a child to run the left command, waits for it, and then runs the right command.
- For `BACK`, it forks a child to run the command and immediately returns without waiting.

4.3 Built-in cd handling and process boundaries

Lines 166–170 of `user/sh.c` reveal that `cd` is explicitly intercepted in the main shell loop before any call to `fork()` (Figure 4.3).

Figure 4.3 - Parent-Process Handling of cd Built-in

```
ubuntu@faf-os-labs:~$ grep -n -C 5 "chdir" user/sh.c
164:    if (*cmd == '\n') // is a blank command
165:      continue;
166:    if(cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' '){
167:      // Chdir must be called by the parent, not the child.
168:      cmd[strlen(cmd)-1] = 0;  // chop \n
169:      if(chdir(cmd+3) < 0)
170:        fprintf(2, "cannot cd %s\n", cmd+3);
171:    } else {
```

Figure 4.3 – Parent-process handling of cd built-in in user/sh.c.

4.4 Answers to Part 4 Observe questions

a) **Name the five kinds of command the xv6 shell knows and the character that produces each.**
`EXEC` (basic command, no special character), `REDIR` (produced by < and >), `PIPE` (produced by `|`), `LIST` (produced by `;`), and `BACK` (produced by `&`).

b) **Where is `cd` handled, and why is it not run as a separate program?**
`cd` is handled in `main()` of `user/sh.c` prior to the `fork()` call. It cannot be run as a separate program because each child process operates in its own isolated address space and process descriptor table. If `cd` were an external program, only the child process would change its directory; upon child exit, the parent shell would remain in the original directory. Therefore, `chdir()` must execute directly in the parent shell process.

c) **Search `sh.c` for any code that handles `*` or `$`. What does that explain about Part 2?**
Searching `sh.c` reveals zero code handling `*` or `$`. Neither globbing nor variable expansion logic exists in the parser. This explains why `echo $HOME` and `ls *` print literal characters in xv6: the shell treats them as ordinary alphanumeric tokens and passes them uninspected to `exec()`.

CONCLUSIONS

Laboratory 4 illuminated the division of responsibilities between user-space shells and operating system kernels. By comparing xv6's 500-line shell with GNU Bash, the essential nature of UNIX design was demonstrated: the kernel supplies basic abstractions (processes, file descriptors, pipes, address spaces), while user-space shells provide convenience features (expansions, environment resolution, readline history). Crucially, empirical testing with `args` proved that quoting and wildcard expansions are lexical transformations executed entirely in user space prior to system call dispatch. Finally, studying `user/sh.c` underscored why operations altering process state (such as `cd`) must execute as shell built-ins rather than external binaries.

REFERENCES

1. Cox, R., Kaashoek, M. F., & Morris, R. (2023). *xv6: a simple, Unix-like teaching operating system*. MIT CSAIL.

2. Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design RISC-V Edition*. Morgan Kaufmann.

3. Love, R. (2013). *Linux System Programming: Talking Directly to the Kernel and C Library* (2nd ed.). O'Reilly Media.

4. Bach, M. J. (1986). *The Design of the UNIX Operating System*. Prentice-Hall.